AI-Driven Expert Medical Diagnostic System

Introduction

The **Hybrid Expert Diagnostic System** is designed to provide patients with a two-pronged medical evaluation based on body temperature and the duration of symptoms. This system uniquely combines two AI paradigms:

- **Symbolic AI (Prolog):** Used for rule-based diagnosis and generating treatment recommendations.
- **Machine Learning (ML):** Utilized for probabilistic risk assessment (Low, Medium, or High risk).

The primary goal is to demonstrate how the logical inference of rule-based systems can work alongside the pattern recognition power of machine learning.

Problem Statement

Modern diagnostics often require both data-driven risk assessments and precise clinical guidance based on expert rules. This project addresses the challenge of creating a cohesive system that:

1. Accepts simple numerical patient inputs (temperature and symptom duration).
2. Provides a quick, data-driven Risk Category evaluation.
3. Uses expert rules to deduce a conclusive diagnosis and treatment plan.
4. Displays all findings through a user-friendly Graphical User Interface (GUI).

Technology Stack

- **Language:** Python 3.x (Application Logic & GUI), Prolog (Knowledge Base).
- **Libraries:** tkinter (GUI), pyswip (Python-Prolog bridge), scikit-learn (ML), numpy, and joblib.
- **External Dependency:** Requires **SWI-Prolog** installed on the environment for the pyswip library to function.

System Architecture

The system follows a hybrid architecture consisting of four main layers:

1. **Presentation Layer (app.py):** A Tkinter-based GUI for user input and result display.
2. **Controller Layer (app.py):** Acts as a bridge between the GUI, the ML service, and the Prolog engine.
3. **Machine Learning Service (classifier_model.py):** Handles training, saving, and executing the Decision Tree model for risk prediction.
4. **Expert System Engine (rules.pl):** The knowledge base and inference engine used for symbolic diagnosis.

Functional Requirements

- **FR1:** Accept Body Temperature (°C) and Symptom Duration (Days).

- **FR2:** Validate inputs (Temperature ≥ 35°C, Duration > 0).
- **FR3:** Automatically train a Decision Tree classifier if the model file is missing.
- **FR4:** Predict a Risk Category (Low, Medium, High) using the ML model.
- **FR5:** Translate numerical data into symbolic Prolog facts based on defined thresholds:
    - **High Fever:** Temp ≥ 39.0°C.
    - **Low Fever:** 37.5°C ≤ Temp < 39.0°C.
    - **Long Duration:** Duration ≥ 5 days.
- **FR6:** Determine a Diagnosis via the Prolog engine (e.g., severe infection, mild viral fever).
- **FR7:** Retrieve a specific Treatment Plan based on the diagnosis.

Core Files

- **app.py**: The main entry point containing the GUI and integration logic.
- **classifier_model.py**: Contains the machine learning model logic.
- **rules.pl**: The Prolog knowledge base containing medical rules.
- **symptoms_data.csv**: The dataset used for training the ML model.
- **requirements.txt**: Lists the necessary Python libraries.

Future Enhancements

- Implementing more complex ML models like SVM or Logistic Regression.
- Expanding the symptom list to include discrete inputs like cough, headache, or fatigue.
- Integrating fuzzy logic in Prolog to handle uncertain or partial diagnoses.
- Migrating the GUI to a web-based framework like Flask or Django for better accessibility.